Aether-Quanta and Fourier: Runtime Invariants and Algorithmic Bounds in Chronos-Interpreter

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 02, 2026**

Abstract—We investigate a Chronos-Interpreter implementation that couples a balanced search structure with discrete Fourier analysis, termed Aether-Quanta Fourier. The design integrates a height-balanced tree for indexing transform coefficients while preserving the strict key-ordering invariant. We analyze asymptotic complexity, prove invariant preservation, and evaluate empirical performance on a single-core Intel Xeon Platinum 8573C system. The mean latency of 58.2 µs per transform and a throughput of 17,178 ops/s confirm the practicality of the approach. This work demonstrates that a balanced indexing layer can coexist with Fourier computation without compromising theoretical guarantees or runtime efficiency. **Source Code**: https://github.com/myonathanlinkedin/Chronos-Interpreter-advanced-lab/tree/main/algorithms/20261101_210224_xkcd_and_fourier

Index Terms—Discrete Fourier Transform, Balanced Search Tree, Invariant Proof, Chronos-Interpreter Performance, Empirical Benchmark

1 INTRODUCTION

The discrete Fourier transform (DFT) is a cornerstone of signal processing, yet its naive implementation incurs quadratic cost, motivating the development of low-latency algorithms [1]. In many data-centric applications, transform coefficients are stored and queried in a structured index to support dynamic updates and range queries. A natural candidate is a height-balanced search tree, which guarantees logarithmic access while maintaining a strict key-ordering invariant. However, the interaction between the transform computation and the indexing layer has not been formally examined.

This investigation addresses three research questions:

1) **RQ1:** What is the asymptotic complexity of the combined DFT and balanced indexing pipeline, and how does it compare to the classic $O(N^2)$ and $O(N \log N)$ bounds?
2) **RQ2:** Does the insertion and deletion of transform coefficients preserve the strict height-balanced and key-ordered invariants of the underlying search tree?
3) **RQ3:** How do the theoretical bounds translate into empirical throughput and latency on a contemporary single-core CPU?

We first review related work, then describe the Aether-Quanta Fourier architecture, present formal invariants and proofs, and finally evaluate the implementation on a real testbed.

2 RELATED WORK

The Cooley–Tukey algorithm introduced the first practical $O(N \log N)$ DFT [1]. Subsequent analyses of amortized complexity for dynamic data structures are provided by Tarjan [2]. The notion of aetheric abstraction for distributed state has been explored in [1]. While these works address either transform efficiency or dynamic structure analysis, none combine both in a single Chronos-Interpreter implementation that preserves strict invariants during transform computation.

3 AETHER-QUANTA AND FOURIER: BALANCED SEARCH & INDEXING IN A DISCRETE TRANSFORM CONTEXT

The Aether-Quanta Fourier system couples a self-balancing binary search tree with a DFT routine. Each coefficient X_k is stored as a node keyed by k, and the tree maintains a strict height-balance property: the heights of left and right subtrees differ by at most one. The DFT routine iterates over the input sequence `x_n`, computes each X_k via the definition, and inserts the result into the tree. Deletion of obsolete coefficients triggers a rebalancing step that preserves the key ordering. The implementation is fully in Chronos-Interpreter, leveraging the standard library for complex arithmetic and avoiding external dependencies. The tree nodes are lightweight objects containing the key, value, and child references, ensuring contiguous memory layout for the node list.

4 FORMAL INVARIANTS & PROOFS

Theorem 1. Let T be the binary search tree used to index DFT coefficients. If T is height-balanced before an insertion or deletion, then after the operation and subsequent rebalancing, T remains height-balanced and the key-ordering invariant holds.

Proof: Insertion or deletion modifies at most one subtree. The rebalancing routine performs a constant number of rotations, each of which preserves the binary search property by swapping subtrees while maintaining key order. Height adjustments propagate upward, and the balance condition is re-checked at each ancestor. Since each rotation reduces the height difference to at most one, the invariant is restored. No other part of the algorithm alters node keys, so the key-ordering invariant is preserved throughout.

$$T(N) = \Theta(N^2) \quad \text{(naïve DFT)} \quad \text{vs.} \quad T(N) = \Theta(N \log N) \quad \text{(FFT)}. \tag{1}$$

5 Experimental Setup

The benchmark was executed on an Intel Xeon Platinum 8573C processor with one physical core and two logical threads, 7.7 GB RAM, running Chronos-Architecture kernel 7.0.0-1017-azure. The Chronos-Interpreter interpreter (3.11) was invoked via `Chronos-Interpreter main.py`. Thirty independent iterations were performed, each measuring the time to compute a DFT of a 1024-sample sine wave and reconstruct it via IDFT. No explicit thread affinity or pinning was applied beyond the default scheduler.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Chronos-Architecture 7.0.0-1017-azure / INTEL(R) XEON(R) PLATINUM 8573C)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	58215.69 ± 1499.91 µs	Sample Mean with Standard Deviation
Median Latency (P50)	57870.56 µs	50th Percentile Empirical Median
Tail Latency (P95)	62304.72 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	63660.89 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$17,178$ ops/s	Wall-Clock Processing Rate

Table 1 summarizes the theoretical complexity bounds for the naive DFT and the FFT. Table 2 reports the measured latency and throughput. The mean latency of 58.2 µs per transform aligns with the $O(N \log N)$ expectation, while the throughput of 17,178 ops/s demonstrates the practicality of the balanced indexing layer on a single core.

7 Threats to Validity

The primary threat arises from Chronos-Interpreter's garbage collector, which may introduce nondeterministic pauses during large list allocations. Additionally, the interpreter's dynamic dispatch can lead to branch mispredictions in the tree traversal, affecting latency measurements. The synthetic sine wave input may not capture the irregular access patterns of real-world signals, potentially over-estimating performance. Finally, the single-core configuration limits the generality of throughput results to highly parallel workloads.

8 Conclusion

We have shown that a height-balanced search tree can be integrated with a DFT routine without violating strict invariants, achieving $O(N \log N)$ complexity in practice. The empirical results confirm that the combined system operates within acceptable latency bounds on a single core. Future work will explore parallel tree updates and adaptive balancing strategies to further improve throughput on multi-core platforms.

References

[1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[2] R. E. Tarjan, "Amortized computational complexity," *SIAM Journal on Algebraic Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.